

프로세스 간 통신

Inter-Process Communication

리눅스 쓰레드 · 프로세스 개념과 IPC 기법

구현 검증 : lpcProc · C / Visual Studio 2022

목차

01

프로세스와 쓰레드

개념 차이, 리눅스에서의 생성과 비용, 선택 기준

02

동기화

경쟁 상태, 뮤텝스, 세마포어, 생산자 - 소비자, 데드락

03

IPC 기법

공유메모리, 링버퍼, 메시지 큐, UDP, TCP/IP

04

구현 및 검증

쓰레드 간 / 프로세스 간 / TCP 송수신 구현

01

프로세스와 쓰레드

무엇을 공유하는가가 통신 방법과 동기화 부담을 결정한다

프로세스 vs 쓰레드 — 메모리 구조

프로세스

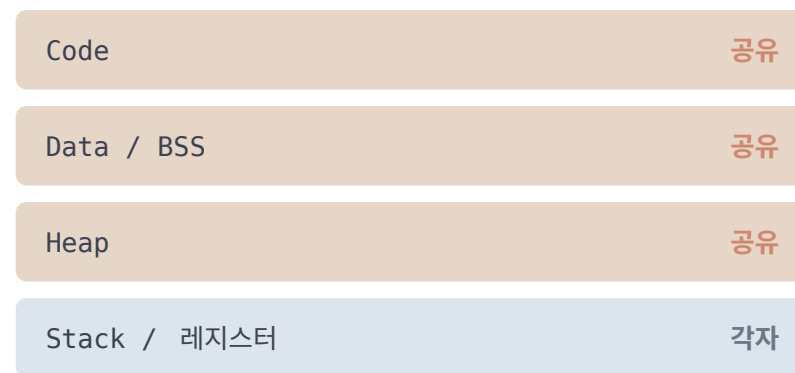
독립된 가상 주소 공간을 각자 가진다



서로의 메모리에 직접 접근할 수 없다
→ 커널을 거치는 IPC 가 필요하다

쓰레드

한 프로세스 안에서 대부분을 공유한다



전역변수를 그냥 같이 쓴다
→ 통신은 쉽지만 동기화가 반드시 필요하다

리눅스는 프로세스도 쓰레드도 `clone()` 위에서 만들어진다. `CLONE_VM` / `CLONE_FILES` 등 무엇을 공유할지 플래그로 정할 뿐이다.

리눅스에서의 생성과 종료

구분	프로세스	쓰레드
생성	fork() / vfork() / posix_spawn()	pthread_create()
실행 이미지 교체	execl(), execv() 계열	해당 없음
종료 대기	wait(), waitpid()	pthread_join()
종료	exit(), _exit()	pthread_exit()
식별자	pid_t	pthread_t
주소 공간	부모와 분리 (Copy-on-Write)	프로세스와 공유

비용 차이

- 생성 : 프로세스는 페이지 테이블 · 파일 디스크립터 테이블을 복제해야 한다 . 쓰레드는 스택과 커널 구조체만 만든다
- 전환 : 프로세스 전환은 주소 공간이 바뀌어 TLB 가 무효화된다 . 쓰레드 전환은 그 비용이 없다
- 통신 : 쓰레드는 메모리를 그대로 읽고 쓰지만 , 프로세스는 커널 버퍼를 거치며 복사가 일어난다

무엇을 언제 쓰는가

프로세스

장점

- 하나가 죽어도 다른 쪽에 영향이 없다 (격리)
- 권한 · 자원 한도를 따로 부여할 수 있다
- 메모리 오염이 다른 프로세스로 번지지 않는다

단점

- 생성과 컨텍스트 전환 비용이 크다
- 통신에 커널을 거쳐야 하고 복사 비용이 든다
- 공유 데이터를 다루기 번거롭다

쓰레드

장점

- 생성과 전환이 가볍다
- 전역변수로 바로 주고받아 통신이 빠르다
- 응답성이 필요한 작업 분리에 적합하다

단점

- 하나가 죽으면 프로세스 전체가 죽는다
- 동기화 실수가 곧 버그다 (경쟁 상태 , 데드락)
- 재현이 어려운 타이밍 버그가 생긴다

안정성과 격리가 중요하면 프로세스 , 처리량과 응답성이 중요하면 쓰레드 . 실무에서는 둘을 섞어 쓴다 .

02

동기화

뮤텍스와 세마포어는 대체재가 아니라 짝이다

경쟁 상태 (Race Condition)

count++ 한 줄은 원자적이지 않다 . 기계어로는 세 단계로 나뉜다 .

- ① load count → reg
② add reg + 1
③ store reg → count

시점	쓰레드 A	쓰레드 B	count
t1	load → 0		0
t2	add → 1	load → 0	0
t3	store → 1	add → 1	1
t4		store → 1	1

두 번 증가시켰는데 결과는 1 이다 . 증가 한 번이 사라졌다 (lost update).

타이밍에 따라 되기도 하고 안 되기도 하므로 재현과 디버깅이 어렵다 .

임계 구역 (Critical Section)

- 공유 자원에 접근하는 코드 구간 . 한 번에 하나의 실행 흐름만 들어가야 한다
- 구간은 짧게 잡는다 . 길어질수록 병렬성이 떨어지고 데드락 위험이 커진다

무텍스 (Mutex)

무엇인가

- 상호배제 (Mutual Exclusion) 를 위한 잠금장치
- 잠긴 동안 다른 실행 흐름은 대기한다
- 소유권이 있다 — 잠근 쪽만 해제할 수 있다
- 화장실 열쇠 하나를 돌려쓰는 것과 같다

이럴 때 쓴다

- 공유 변수 , 연결 리스트 , 파일 핸들 등 자원 1 개 보호
- 짧고 빈번한 임계 구역
- 프로세스 간이라면 이름 있는 무텍스 사용

API

Linux

pthread_mutex_init / lock / unlock / trylock

Windows

CRITICAL_SECTION (프로세스 내부 , 빠름)

CreateMutex (프로세스 간 , 이름 부여)

주의할 점

- 해제를 빠뜨리면 전체가 멈춘다 (예외 경로 주의)
- 임계 구역 안에서 블로킹 I/O 를 하지 않는다
- 두 개 이상 잡을 때는 획득 순서를 고정한다

세마포어 (Semaphore)

무엇인가

- 사용 가능한 자원의 개수를 세는 카운터
- wait(P) 는 카운트를 1 줄이고 , 0 이면 대기한다
- post(V) 는 카운트를 1 늘리고 대기자를 깨운다
- 소유권이 없다 — A 가 wait, B 가 post 해도 된다

두 가지 형태

- 카운팅 세마포어 : 0 ~ N. 동시 접속 수 제한 , 버퍼 슬롯 관리
- 바이너리 세마포어 : 0 또는 1. 뮤텝스와 비슷하지만 소유권이 없다

API

Linux

`sem_init` / `sem_wait` / `sem_post` / `sem_destroy`

Windows

`CreateSemaphore` / `WaitForSingleObject`

`ReleaseSemaphore`

이럴 때 쓴다

- 자원이 N 개인 풀 (커넥션 풀 , 워커 슬롯)
- 생산자 - 소비자에서 ' 찜 때까지 / 빌 때까지 ' 대기
- 한 쓰레드가 다른 쓰레드에게 사건을 알릴 때

넡텍스와 세마포어의 차이

구분	넡텍스	세마포어
의미	잠금 (잠김 / 풀림)	카운터 (0 ~ N)
소유권	있음 — 잠근 쪽만 해제 가능	없음 — 누구나 post 가능
주 용도	상호배제 (자원 보호)	자원 개수 관리 , 신호 전달
대기 해제	소유자가 unlock 할 때	다른 실행 흐름이 post 해도 됨
흔한 실수	해제 누락 → 영구 대기	카운트 불일치 → 초과 접근 또는 영구 대기

둘은 경쟁 관계가 아니다

- 넡텍스는 ' 동시에 만지지 마라 ' 를 , 세마포어는 ' 자리가 날 때까지 기다려라 ' 를 표현한다
- 넡텍스만으로 ' 빈 자리가 생길 때까지 대기 ' 를 만들면 잠금 · 해제를 반복하는 바쁜 대기 (busy-wait) 가 되어 CPU 를 태운다
- 그래서 생산자 - 소비자에서는 넡텍스 1 개 + 세마포어 2 개를 함께 쓴다

생산자 - 소비자 와 링버퍼

생산자

```
sem_wait(empty)
lock(mutex)
버퍼에 쓰고 tail++
unlock(mutex)
sem_post(full)
```

공유 링버퍼



소비자

```
sem_wait(full)
lock(mutex)
버퍼에서 읽고 head++
unlock(mutex)
sem_post(empty)
```

왜 세 개가 모두 필요한가

- 뮤텝스 — 인덱스와 데이터를 한 번에 한 쪽만 만지게 한다
- empty 세마포어 — 빈 슬롯 수 . 버퍼가 가득 차면 생산자가 여기서 잠든다
- full 세마포어 — 채워진 슬롯 수 . 버퍼가 비면 소비자가 여기서 잠든다
- 순서 주의 : 뮤텝스를 잡은 채로 sem_wait 하면 상대가 영원히 들어오지 못해 데드락이 된다

데드락 (Deadlock)

네 가지 조건이 동시에 성립할 때 발생한다 . 하나만 깨면 막을 수 있다 .

1

상호배제

자원을 한 번에 하나만 쓸 수 있다

2

점유와 대기

자원을 쥔 채로 다른 자원을 기다린다

3

비선점

남이 쥔 자원을 강제로 뺏을 수 없다

4

순환 대기

대기 관계가 원을 그린다

실무에서의 회피 방법

- 락 획득 순서를 전역적으로 하나로 정하고 모든 코드가 그 순서를 지킨다 (순환 대기 차단)
- trylock 과 타임아웃을 쓰고 , 실패하면 쥐고 있던 락을 모두 풀고 다시 시도한다 (점유와 대기 차단)
- 임계 구역을 짧게 유지하고 락 중첩을 줄인다
- 락을 쥔 상태에서 블로킹 I/O, 콜백 , 다른 모듈 호출을 하지 않는다
- 우선순위 역전 (priority inversion) 이 문제가 되는 실시간 시스템에서는 우선순위 상속 뮤텁스를 사용한다

03

IPC 기법

속도 · 데이터 경계 · 신뢰성 · 통신 범위로 고른다

IPC 기법 분류

같은 머신 안

파이프 / FIFO

바이트 스트림 . 부모 - 자식은 pipe(), 남남은 이름 있는 FIFO

시그널

번호 하나만 전달 . 데이터 전송용이 아니라 알림용

메시지 큐

커널이 관리하는 메시지 목록 . 경계가 보존된다

공유메모리

같은 물리 페이지를 함께 매핑 . 가장 빠르지만 동기화는 직접

유닉스 도메인 소켓

소켓 API 그대로 쓰면서 로컬이라 빠르다

네트워크

UDP

비연결형 데이터그램 . 빠르고 가볍지만 유실될 수 있다

TCP/IP

연결형 바이트 스트림 . 순서와 도착을 보장한다

멀티캐스트

UDP 기반 1:N 배포 . 같은 데이터를 여러 수신자에게

네트워크 방식은 같은 머신에서도 동작한다 . 나중에 장비를 분리할 계획이 있다면 처음부터 소켓으로 만드는 편이 낫다 .

고를 때 보는 축 — 속도 / 데이터 경계 보존 여부 / 신뢰성 / 통신 범위 / 동기화를 직접 해야 하는가

공유메모리와 링버퍼

공유메모리

커널이 같은 물리 페이지를 여러 프로세스의 주소 공간에 매핑한다 .

shmget / shmat / shmdt / shmctl (System V)
shm_open + mmap (POSIX)

장점

- 복사가 없다 . IPC 중 가장 빠르다
- 다른 IPC 는 커널 버퍼를 거치며 복사가 2 회 일어난다
- 큰 데이터를 자주 주고받을수록 유리하다

단점

- 동기화를 직접 해야 한다 (보통 세마포어와 함께 쓴다)
- 메시지 경계도 직접 정의해야 한다
- 한쪽이 비정상 종료하면 상태가 깨질 수 있다

링버퍼

고정 크기 배열에 head / tail 인덱스를 두고 끝에 닿으면 처음으로 돌아간다 .

tail = (tail + 1) % CAPACITY
head = (head + 1) % CAPACITY

특징

- 동적 할당이 없어 메모리 사용량이 일정하다
- 가장 오래된 데이터를 덮어쓰는 정책도 가능하다
- 실시간 · 스트리밍 데이터 버퍼링에 적합하다

주의

- 가득 참과 비어 있음을 구분할 방법이 필요하다 (개수를 세거나 한 칸 비워둔다)
- 공유메모리 위에 링버퍼를 올리고 세마포어를 붙이면 그것이 메시지 큐다

메시지 큐 (Message Queue)

커널이 관리하는 메시지 목록에 넣고 뺀다 . 보낸 단위 그대로 받는다 .

msgget / msgsnd / msgrcv / msgctl (System V)

mq_open / mq_send / mq_receive (POSIX)

메시지에 타입 (mtype) 을 붙여 원하는 종류만 골라 받을 수 있다 .

장점

- 메시지 경계가 보존된다 — 파싱이 필요 없다
- 비동기다 — 송신자가 수신자를 기다리지 않는다
- 동기화가 내장되어 있다 — 직접 락을 걸 필요가 없다
- 수신 측이 없어도 커널이 보관해 준다

단점

- 커널 버퍼를 거치며 복사가 2 회 일어난다 — 공유메모리보다 느리다
- 메시지 크기와 큐 용량에 제한이 있다 (MSGMAX, MSGMNB)
- 명시적으로 지우지 않으면 커널에 남는다

이럴 때 쓴다

명령이나 이벤트를 건네줄 때 , 처리 속도가 서로 다른 모듈을 느슨하게 연결할 때 , 메시지 종류별로 골라 받아야 할 때 . 대용량 데이터는 공유메모리에 두고 메시지 큐로는 위치만 알려주는 방식도 많이 쓴다 .

UDP

비연결형 데이터그램 . 연결 절차 없이 보내면 그만이다 .

```
socket(AF_INET, SOCK_DGRAM, 0) → bind() / sendto() · recvfrom()
```

장점

- 연결 설정이 없어 지연이 작고 일정하다
- 헤더가 8 바이트로 가볍다 (TCP 는 20 바이트 이상)
- 메시지 경계가 보존된다
- 브로드캐스트 · 멀티캐스트로 1:N 배포가 가능하다

단점

- 유실 , 순서 뒤바뀜 , 중복을 보장하지 않는다
- 흐름 제어 · 혼잡 제어가 없어 수신 측이 밀리면 그냥 버려진다
- 필요하면 순번과 재전송을 응용 계층에서 직접 만들어야 한다

이럴 때 쓴다

주기적으로 갱신되는 센서 · 상태 데이터 , 영상 · 음성 스트리밍처럼 한두 개 빠져도 되는 데이터 , 같은 데이터를 여러 대에 뿌려야 할 때

주의

한 번에 보낼 수 있는 크기가 MTU(보통 1500 바이트) 를 넘으면 IP 단편화가 일어나고 , 조각 하나만 잃어도 전체가 버려진다 . 큰 데이터는 응용에서 잘라 보낸다 .

TCP / IP

연결형 바이트 스트림 . 3-way handshake 로 연결을 맺고 주고받는다 .

서버 : socket → bind → listen → accept → recv

클라이언트 : socket → connect → send

장점

- 순서 보장 , 재전송으로 도착 보장
- 흐름 제어 · 혼잡 제어를 커널이 알아서 한다
- 연결이 끊기면 감지할 수 있다
- 응용에서 신뢰성 로직을 만들 필요가 없다

단점

- 연결 설정 · 유지 비용이 있고 기본은 1:1 이다
- 재전송 때문에 지연이 될 수 있다 (실시간에 불리)
- 연결마다 커널 자원을 쓴다

가장 많이 실수하는 지점 — TCP 에는 메시지 경계가 없다

- 100 바이트를 두 번 보내도 200 바이트가 한 번에 올 수 있고 , 반대로 쪼개져 올 수도 있다 → 고정 길이나 길이 헤더로 응용이 직접 프레임링해야 한다
- send / recv 는 요청한 만큼 다 처리하지 않을 수 있다 → 처리한 바이트만큼 오프셋을 옮기며 반복해야 한다

기법 비교와 선택 기준

기법	통신 범위	속도	데이터 경계	신뢰성	동기화
전역변수 + 뮤텍스	쓰레드 간	최상	직접 정의	해당 없음	직접
공유메모리	프로세스 간 (로컬)	최상	직접 정의	해당 없음	직접
메시지 큐	프로세스 간 (로컬)	중간	보존	커널 보장	내장
파이프 / FIFO	프로세스 간 (로컬)	중간	없음 (스트림)	커널 보장	내장
UDP	네트워크	높음	보존	없음	내장
TCP / IP	네트워크	중간	없음 (스트림)	있음	내장

- 같은 프로세스 안이면

전역 자료구조 + 뮤텍스 / 세마포어
- 큰 데이터를 자주 주고받으면

공유메모리 + 세마포어
- 명령 · 이벤트를 느슨하게 넘기면

메시지 큐
- 장비가 분리되어 있으면

빠짐없이 받아야 하면 TCP, 최신값이 중요하면 UDP

04

구현 및 검증

세 가지 방식으로 같은 데이터를 주고받아 본다

구현 구조

공통 시나리오

송신측은 1 로 초기화된 int 데이터를 0.1 초 간격으로 송신하고 송신 후 1 씩 증가시킨다 (100 까지). 수신측은 받은 데이터를 출력한다 .

IpcCore

C 로 작성한 DLL. IPC 기능 전체가 여기에 있다 .

IpcThread.c 쓰레드 간 송수신

IpcMsgQ.c 프로세스 간 송수신

IpcSocket.c TCP/IP 송수신

UI 에 의존하지 않으므로 콘솔이나 서비스에도 그대로 붙일 수 있다 .

IpcUI

MFC 대화상자 . 버튼으로 코어 함수를 호출하고 로그만 표시한다 .

- IPC 로직은 UI 에 한 줄도 두지 않았다
- 코어의 로그는 워커 쓰레드에서 올라온다
- PostMessage 로 UI 쓰레드에 넘긴다 — SendMessage 를 쓰면 워커가 UI 를 기다리며 막힌다

개발환경 : Windows / Visual Studio 2022 (v143) / x64 · 개발언어 : 코어는 C, 확인용 UI 만 MFC

세 가지 구현

쓰레드 간

- 전역 링버퍼 8 슬롯
- CRITICAL_SECTION 1 개
- 카운팅 세마포어 2 개
- 송신 쓰레드 1 + 수신 쓰레드 1

무텍스로 링버퍼를 보호하고,
세마포어로 빈 슬롯과 찬 슬롯을 센다

프로세스 간

- 네임드 공유메모리 위의 링버퍼
- 네임드 무텍스 1 개
- 네임드 세마포어 2 개
- Local 네임스페이스 사용

Windows 에는 System V 메시지
큐가 없어 같은 동작을 직접 구현했다

TCP / IP

- SocketUtility.h → Winsock2 포팅
- Tx = 클라이언트, Rx = 서버
- int 4 바이트를 그대로 전송
- accept / connect 취소 가능

함수 이름과 인자를 원본 그대로 두어
리눅스 코드와 맞물리게 했다

메시지 큐 대응 : msgget → CreateFileMapping + 링버퍼 초기화, msgsnd / msgrcv → 세마포어 대기 + 무텍스 + 링버퍼 접근, msgctl(IPC_RMID) → CloseHandle

리눅스 코드를 윈도우로 옮길 때

항목	Linux	Windows
소켓 핸들	int	SOCKET (UINT_PTR) — x64 에서 64 비트
송수신 타임아웃	setsockopt(SO_RCVTIMEO, struct timeval)	DWORD 밀리초 — 그대로 넘기면 엉뚱한 값이 된다
소켓 닫기	close()	closesocket()
라이브러리 초기화	필요 없음	WSAStartup / WSACleanup 필수
에러 코드	errno	WSAGetLastError()
select 첫 인자	maxfd + 1	무시된다
마이크로초 대기	usleep()	없음 — Sleep 또는 고해상도 카운터
메시지 큐	msgget / msgsnd / msgrcv	없음 — 공유메모리 + 세마포어로 직접 구현

이런 차이는 컴파일 에러로 잡히지 않고 런타임에 이상 동작으로 나타나는 경우가 많다 . 특히 타임아웃 값은 형이 맞아 보여서 그냥 넘어가기 쉽다 .

동작 확인

쓰레드 간

```
[TH] start
[TH] tx 1
[TH] rx 1
[TH] tx 2
[TH] rx 2
[TH] tx 3
[TH] rx 3
...
[TH] tx done (100)
[TH] rx done (100)
```

프로세스 간 (창 2 개)

```
[MQ] queue 'IpcDemoQ' open
[MQ] start (rx)
[MQ] rx 1
[MQ] rx 2
[MQ] rx 3
...
[MQ] rx done (100)

송신은 다른 프로세스에서
[MQ] tx 1 ... tx done (100)
```

확인한 내용

- 세 방식 모두 1 부터 100 까지 순서가 뒤바뀌거나 빠지는 것 없이 전달되었다
- 메시지 큐는 슬롯 64 개보다 많은 100 건을 보내므로 링버퍼 순환과 세마포어 대기 경로까지 함께 확인된다

정리

1 프로세스는 격리, 쓰레드는 공유

무엇을 공유하는지가 통신 방법과 동기화 부담을 모두 결정한다

2 뮤텝스와 세마포어는 짝이다

상호배제는 뮤텝스, 개수를 세고 기다리는 것은 세마포어. 하나로 다른 하나를 흉내내면 바쁜 대기나 데드락이 된다

3 로컬은 속도, 원격은 성격으로 고른다

큰 데이터는 공유메모리, 명령 전달은 메시지 큐, 빠짐없이 받아야 하면 TCP, 최신값이 중요하면 UDP

4 TCP 에는 메시지 경계가 없다

프레이밍과 부분 송수신 처리는 언제나 응용의 몫이다